

Chris Middleton

Due 3/31/25

CMPE 310

The first thing I had to do was figure out the basic functions of this code. In order, the steps to successfully meet requirements was to open the file, read the file, then write the contents of that file line by line into a register (to sum the numbers together). The program would then need to add each line into the register until all numbers in the file had been summed and would finally need to print that sum (and the total amount of lines of numbers in the file, given by the first line of the text file). For this program specifically I used the file given with the project instructions. A large portion of this project was made easier using commands from the C Library. Printf to print text, fopen and fclose for file access, feof to signal the end of the file, and fscanf for reading individual lines in the file.

The first portion of the program initializes the register for printf. Without push rbp and pop rbp, printf causes the program to segfault. I'm unsure as to why, but after some research this was the solution that worked. Then, the pathname variable is read for the file being opened, and rsi is set to read mode (to work in accordance with the C Library). Fopen is then called to open the file, the pointer of which gets stored to the file variable. Next is the function that reads the first line of the file and prints it (which should be the number of lines in the file). The next section focuses on looping through each line of the text file, moving each number into rax. Rbx stores the total, which is added to rax and then moved back to the total variable. This might appear convoluted, but because we're reading each number line by line it's best to store that number in rax before adding it to the total. Feof is then called after the file is scanned. If the end of the file has not been reached, the function repeats. After all numbers have been looped through, the final result is printed using printf. The file is then closed and all registers are reset.

The Code:

extern printf

extern fopen

extern fclose

extern feof

extern fscanf

section .bss

; file: C File pointer, must be a 64 bit variable

file resq 1

; MAX_LENGTH: CONST, max length of number in file

MAX_LENGTH equ 16

; line: char[MAX_LENGTH], buffer to store individual lines

line resb MAX_LENGTH

; number: parsed value of a line

number resb 1

; numCount: number of lines given in the file, though ignored as we use feof

numCount resw 1

section .data

; pathname: name of the file to parse numbers from

pathname dq "randomInt100.txt", 0

; fmt: formatting used when parsing (or printing) numbers

fmt db "%d", 10, 0

; linecount: string used when printing the number of lines in the file (according to the first line)

linecount db "Line count = %d", 10, 0

; totalCount: string used when displaying the sum of the numbers in the file

totalCount db "Total count = %d", 10, 0

; READ_MODE: C file open mode used when reading the file

READ_MODE db "r", 0

; total: stores the sum of all numbers in the file

total dd 0

section .text

global main

main:

; init

push rbp

mov rbp, rsp

; Open file, saves FILE pointer to file variable

mov rdi, pathname

mov rsi, READ_MODE

call fopen

mov [file], rax

read_first:

; Read a number from the file

mov rdi, [file]

mov rsi, fmt

mov rdx, numCount

call fscanf

; print number of lines

```
mov rdi, linecount
mov rsi, [numCount]
xor eax, eax
call printf
```

print_loop:

```
; Read a number from the file
```

```
mov rdi, [file]
```

```
mov rsi, fmt
```

```
mov rdx, number
```

```
call fscanf
```

```
; Uncomment to print the individual lines
```

```
; mov rdi, fmt
```

```
; mov rsi, [number]
```

```
; xor eax, eax
```

```
; call printf
```

```
; Add to the total
```

```
mov rax, [number]
```

```
mov rbx, [total]
```

```
add rax, rbx
```

```
mov [total], rax
```

```
; Loop
```

```
mov rdi, [file]
```

```
call feof ; sets rax to 0 if not end of file
```

```
jz print_loop
```

print_total:

```
; prints the total
```

```
mov rdi, totalCount
```

```
mov rsi, [total]
```

```
xor eax, eax
```

```
call printf
```

close:

```
; Close the file
```

```
mov rdi, [file]
```

```
call fclose
```

```
mov rsp, rbp
```

```
pop rbp
```

```
ret
```

Output:

```
[jz72498@linux6 proj2]$ proj2
Line count = 100
Total count = 4579
```